



Compte-rendu Projet lode runner

Auteur :
Arthur ALLAEYS

Professeur :
François GOASDOUÉ

8 décembre 2024

Table des matières

1	Introduction	2
1.1	Présentation du jeu	2
1.2	Contexte du projet	2
2	Préliminaires	3
2.1	Présentation des moyens algorithmiques employés	3
2.2	Principe des moyens algorithmiques employés	3
3	Solution au problème posé	5
3.1	Principe de la stratégie	5
3.2	Implémentation de la stratégie	7
3.2.1	Parcours en largeur	7
3.2.2	Recherche du bonus le plus proche	10
3.2.3	Reconstruction du chemin	11
3.3	Problèmes rencontrés	12
3.4	Évaluation expérimentale	15
4	Conclusion	16

1 Introduction

1.1 Présentation du jeu

Lode Runner est un jeu vidéo développé par Douglas E. Smith et publié par Brøderbund en 1983. Il connut un succès important au Japon et fut adapté sur de nombreux supports et en différentes versions.

Au départ, le jeu consiste en un personnage, contrôlé par le joueur, qui doit récupérer un trésor disséminé à travers les niveaux sous la forme de lingots. Il va donc évoluer dans des environnements constitués d'échelles, de murs et de passerelles (Figure 1), tout en étant pourchassé par des gardes. Toutefois, le joueur n'est pas sans défense, car il peut créer des trous afin de piéger les ennemis ².

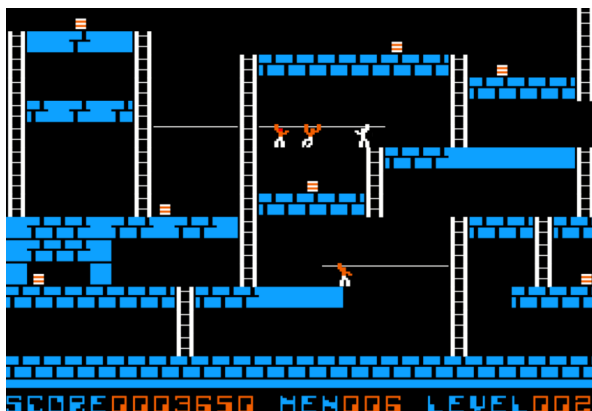


FIGURE 1 – Exemple d'un niveau de Lode Runner ¹. On peut y voir les différentes structures présentes.

1.2 Contexte du projet

Ce document s'inscrit dans le cadre d'un projet du cours d'algorithmique. Il est ici question de la création d'une IA simulant un joueur de lode runner. Les enjeux de ce projet sont donc de concevoir et d'implémenter une IA, afin de gagner à coup sûr à ce jeu. Notre programme devra être capable de réussir cela malgré un manque d'informations (méconnaissance des déplacements ennemis), mais aussi avec la contrainte de ne pas pouvoir mémoriser d'informations entre les tours.

Face à cette difficulté, je ne suis pas parvenu à ce que mon programme soit capable de gagner à tous les coups. Cependant, il parvient à déjouer les embuscades ennemis dans de nombreux cas.

2. [https://fr.wikipedia.org/wiki/Lode_Runner_\(jeu_vidéo,_1983\)](https://fr.wikipedia.org/wiki/Lode_Runner_(jeu_vidéo,_1983))
1. <https://loderunnerwebgame.com/game/>

2 Préliminaires

2.1 Présentation des moyens algorithmiques employés

Dans le cadre de ce projet, j'ai été amené à repartir de zéro pendant la phase de développement. En effet, j'ai tout d'abord essayé d'assimiler la grille de jeu à un graphe et d'en calculer la matrice d'adjacences. Après cela, j'ai tenté de réaliser un parcours en largeur grâce à la matrice d'adjacences, puis de déterminer le chemin le plus court vers un bonus grâce au parcours en largeur, et ce pour chaque bonus afin de trouver le plus proche et le prochain déplacement à réaliser pour y parvenir. Cependant, je ne suis pas parvenu à faire fonctionner cette stratégie et j'ai donc décidé de m'orienter vers une autre plus simple.

Remarque :

Une motivation supplémentaire à ce "retour à zéro" a été la médiocrité algorithmique de cette stratégie. En effet, de nombreux modules peu optimisés ont été nécessaires pour simplement trouver le plus court chemin vers le bonus le plus proche, là où cette étape de ma nouvelle stratégie est beaucoup plus efficace est concise en termes de lignes de codes. De plus, le calcul et l'utilisation de la matrice d'adjacences pour le parcours en largeur avaient une complexité spatiale et temporelle peu satisfaisante. Le code de cette première tentative a été commenté et placé en fin de fichier.

Ma stratégie finale repose finalement sur un parcours en largeur réalisé directement sur la grille de jeu. Puis, grâce à ce parcours en largeur, je détermine quel bonus est le plus proche, et quel est le prochain mouvement à réaliser pour s'en approcher.

Remarque :

Il doit y avoir une erreur ou maladresse dans mon parcours en largeur. En effet, le joueur monte l'échelle menant à la sortie sans raison apparente. Mis à part ce point, le joueur emprunte toujours le plus court chemin vers le bonus le plus proche.

2.2 Principe des moyens algorithmiques employés

Le parcours en largeur :

Le parcours en largeur, aussi appelé BFS pour Breadth-First Search en Anglais, permet de parcourir un graphe non pondéré (ici également non orienté). Ici, on peut assimiler les plateformes et échelles à des noeuds (en pratique également le vide pour pouvoir sauter), et les "espaces entre les cases" à des arrêtes.

Le principe est le suivant : on commence par explorer un noeud soucre, ici là où le joueur se trouve. On liste ensuite les voisins de ce noeuds qui n'ont pas encore été explorés, avant de les explorer à leur tour. Pour ce faire, on utilise une file, en ajoutant

à chaque fois les voisins non explorés à la fin de cette dernière. Ceci permet de ne pas explorer plusieurs fois un noeud, et garantit de voir les sommets par distance croissance à la source. Cette propriété du parcours en largeur permet de résoudre des problèmes de plus court chemin. Enfin, le parcours en largeur a une complexité temporelle en $\mathcal{O}(|S| + |A|)$, ce qui est relativement correct.

La file :

Une file est un type abstrait de structure de données. Cette structure est basée sur le principe FIFO pour "First In, First Out" en Anglais, ou "premier entré, premier sorti" en Français. C'est-à-dire que les premiers éléments ajoutés (enfilés) seront les premiers à être retirés (défilés). C'est le principe de la file d'attente.

J'ai choisi d'implémenter ma file par un tableau alloué dynamiquement et faisant la taille de la grille, afin d'être sûr d'avoir suffisamment de place. J'ai également utilisé deux indices entiers, afin de connaître la position du début et de la fin de la file dans le tableau.

3 Solution au problème posé

3.1 Principe de la stratégie

Ma stratégie repose sur une gestion des ennemis en temps réel. Cela signifie que, tout le temps que le ou les ennemis sont trop loin pour poser un danger au joueur, l'IA va jouer comme s'il n'y avait pas d'ennemis. L'IA ne va donc pas modifier son parcours, et cherchera toujours à emprunter le chemin le plus court malgré la présence possible d'ennemis. J'applique une stratégie qui consiste à éliminer les ennemis sur mon passage, je ne cherche pas à calculer d'autres itinéraires moins dangereux. Je vais donc détailler les différents cas de figure pouvant survenir.

— Présence d'un ennemi à gauche/droite sur une plateforme :

On pose $(x; y)$ la position du joueur. Supposons que l'ennemi se trouve à gauche (respectivement à droite) du joueur. Deux cas de figure se présentent.

1. L'ennemi se trouve en $x-1$ (respectivement $x+1$) :
Dans ce cas, le joueur va se déplacer d'une case vers la gauche (respectivement vers la droite). En effet, il n'a pas le temps de poser une bombe pour éliminer l'ennemi, qui est déjà trop près.
2. L'ennemi se trouve en $x-2$ (respectivement en $x+2$) ou en $x-1$ (respectivement en $x+1$) sur une échelle :
Dans ce cas, si l'ennemi est effectivement en haut ou au pied d'une échelle, le joueur ne peut pas poser de bombe et donc le premier cas ne peut pas s'appliquer. Le joueur va donc se déplacer vers la gauche (respectivement vers la droite). Sinon, il y a une case libre entre le joueur et l'ennemi, et il est donc possible de poser une bombe pour l'éliminer.
3. Le câble peut être relié à une échelle :
Dans ce cas, lorsque l'on arrive au bout du câble, il faut vérifier qu'il n'y ait pas d'ennemi sur l'échelle, qui puisse nous atteindre au prochain tour. On vérifie donc les positions $(x - 1; y - 1)$, $(x - 1; y + 1)$, $(x + 1; y - 1)$ et $(x + 1; y + 1)$.

— Présence d'un ou plusieurs ennemis sur un câble :

Le joueur se trouve sur un câble. On pose x sa position en abscisse. Plusieurs cas de figure se présentent.

1. Il y a un ennemi en $x-1$ ou $x-2$ (respectivement en $x+1$ ou $x+2$) :
Dans ce cas, le joueur va se déplacer dans la direction opposée. De cet façon, il finira par arriver sur une plateforme et pourra éliminer l'ennemi comme expliqué au premier tiret.
2. Il y a un ennemi des deux côtés (en $x-2$ ou $x-1$ et en $x+1$ ou $x+2$) :
Dans ce cas, le joueur est encerclé et n'a pas d'autre choix que de sauter.

— Un ennemi arrive sur la plateforme par une échelle :

Jusqu'à présent, je me souciais uniquement des ennemis de même ordonnée que le joueur. Cependant, il peut arriver que l'ennemi arrive par une échelle sur la plateforme sur laquelle se trouve le joueur. C'est une menace qui n'est pas prise en compte par le premier tiret, et qui doit être traitée. On pose $(x; y)$ la position du joueur.

1. L'ennemi descend l'échelle à gauche du joueur (respectivement à droite) :
Si un ennemi se trouve en $(x-1; y-1)$, respectivement en $(x+1; y-1)$, le joueur va se déplacer vers la droite en $(x+1; y)$, respectivement vers la gauche en $(x-1; y)$ afin de rentrer dans la configuration du premier tiret.
2. L'ennemi monte l'échelle à gauche du joueur (respectivement à droite) :
Si un ennemi se trouve en $(x-1; y+1)$, respectivement en $(x+1; y+1)$, sur une échelle, alors on procède de même qu'au premier point. Il faut ici vérifier que l'ennemi est sur une échelle, car il peut être dans un trou de bombe. Il est en effet possible de passer sur l'ennemi, qui bouche temporairement le trou. Si l'on ne le vérifiait pas, le joueur ne passerait pas au dessus du trou et s'exposerait à la menace d'un ennemi arrivant de l'autre côté.

— Présence d'un ou plusieurs ennemis sur une échelle ou en sortie de l'échelle :

Le joueur se trouve sur une échelle. On pose $(x; y)$ sa position. Plusieurs cas de figure se présentent.

1. Il y a un ennemi au dessus :
On vérifie d'abord si l'ennemi est sur l'échelle, c'est-à-dire en $(x; y-1)$ ou $(x; y-2)$. Cependant, on pourrait arriver en haut de l'échelle et se retrouver face à un ennemi. On vérifie donc si un ennemi se trouve en $(x-2; y-1)$, $(x-1; y-1)$, $(x+1; y-1)$ ou $(x+2; y-1)$. Si un ennemi a été détecté à une de ces positions, alors le joueur descend en $(x; y+1)$.
2. Il y a un ennemi en dessous :
On procède de même, en vérifiant les positions : $(x; y+1)$, $(x; y+2)$, $(x-1; y+1)$ et $(x+1; y+1)$. Néanmoins, il faut vérifier pour la position $(x; y+2)$, qu'il s'agit bien de l'emplacement d'une échelle. En effet, le joueur pourrait se trouver au pied de l'échelle, et ainsi détecter un ennemi se trouvant sur une plateforme en dessous. De plus, il faut vérifier pour les positions $(x-1; y+1)$ et $(x+1; y+1)$, que l'ennemi se trouve bien sur une plateforme ou qu'il a un câble au dessus de lui. En effet, il est apparu que si le joueur élimine un ennemi grâce à une bombe, puis prend immédiatement l'échelle et qu'un ennemi arrive par en dessous, une boucle infinie survient (Figure 2). Si un ennemi a été détecté à une de ces positions, alors le joueur monte en $(x; y-1)$.
3. Il y a un ennemi au dessus et en dessous :
Si un ennemi est détecté en $(x; y-1)$ ou $(x; y-2)$ et en $(x; y+1)$ ou $(x; y+2)$, alors le joueur est encerclé et n'a pas d'autre choix que de sauter à gauche ou à droite. Par défaut, il essaiera de sauter à droite, et sautera à gauche si ce n'est

pas possible. Il faut donc vérifier si un ennemi se trouve en dessous à gauche ou à droite pour prendre cette décision. On regarde également la case de chaque côté de celle d'atterrissage. En effet, un ennemi juste à côté nous aurait une fois atterri. On vérifie ceci à chaque fois que l'on veut sauter d'une échelle ou d'une plateforme.

3.2 Implémentation de la stratégie

Dans cette partie, je vais tâcher de présenter trois modules clés de ma stratégie. Pour ce faire, j'en ai écrit le pseudo-code.

J'ai choisi ces trois modules car ils forment la base de la stratégie du joueur, et sont suffisants pour le faire jouer. Ils permettent de réaliser les déplacements nécessaires pour aller chercher les bonus et finir le jeu. Ils forment donc la couche de base à laquelle il faut ajouter une seconde couche permettant la gestion des ennemis. J'ai choisi de ne pas détailler le pseudo-code de cette seconde couche, car cela serait redondant avec la partie 3.1.

3.2.1 Parcours en largeur

Pour le pseudo-code de ce module, je note size la variable appelée taille dans le code C, afin de ne pas amener de confusion avec la fonction taille.

```
1 Fonction parcours_largeur en plvalue
2 Déclaration
3   Enregistrements
4     levelinfo(map en tableau à 2 dimensions de caractères,
5     xsize en entier, ysize en entier, xexit en entier,
6     yexit en entier),
7     character(item en caractère, x en entier, y en entier,
8     d en action),
9     position(x en entier, y en entier),
10    plvalue(dist en tableau d'entiers, pred en tableau
11    de positions, actionpred en tableau d'actions)
12
13   Énumérations
14     action={NONE, UP, DOWN, LEFT, RIGHT, BOMB_LEFT,
15     BOMB_RIGHT}
16
17   Variables
18     size en entier, debut en entier, fin en entier, courant
19     en position, distances en tableau d'entiers, file en
20     tableau de positions, predecesseurs en tableau de
21     positions, action_precedente en tableau d'actions
22
```



```

23 Paramètres
24     lvlinfo en levelinfo, joueur en character
25
26 Début
27     size ← lvlinfo.xsize * lvlinfo.ysize;
28     /* Initialisation des structures */
29     info ← allouer(taille(plvalue));
30     file ← allouer(size * taille(position));
31     predecesseurs ← allouer(size * taille(position));
32     action_precedente ← allouer(size * taille(action));
33     Pour i de 0 à taille - 1 Faire :
34         distances[i] ← -1;
35         predecesseurs[i].x ← -1;
36         predecesseurs[i].y ← -1;
37     FinPour
38     debut ← 0;
39     fin ← 0;
40
41     /* Ajouter la position du joueur dans la file */
42     file[debut].x ← joueur.x;
43     file[debut].y ← joueur.y;
44     distances[joueur.y * lvlinfo.xsize + joueur.x] ← 0;
45     fin ← fin + 1;
46
47     /* Parcours en largeur */
48     TantQue debut < fin Faire :
49         /* On dépile le premier sommet de la file */
50         courant ← file[debut];
51         debut ← debut + 1;
52
53         /* Explorer les voisins accessibles */
54
55         /* On enfile le sommet du dessus si possible */
56         Si lvlinfo.map[courant.y - 1][courant.x] = LADDER
57         Alors :
58             Si distances[(courant.y - 1) * lvlinfo.xsize +
59             courant.x] = -1
60             Alors :
61                 file[fin].x ← courant.x;
62                 file[fin].y ← courant.y - 1;
63                 distances[file[fin].y * lvlinfo.xsize +
64                 file[fin].x] ← distances[courant.y *
65                 lvlinfo.xsize + courant.x] + 1;
66                 predecesseurs[file[fin].y * lvlinfo.xsize
67                 + file[fin].x] ← courant;
68                 action_precedente[file[fin].y * lvlinfo.xsize

```

```

69         + file[fin].x] ← UP;
70     fin ← fin + 1;
71     FinSi
72 FinSi
73
74 /* On enfile le sommet du dessous si possible */
75 Si lvlinfo.map[courant.y + 1][courant.x] ∈ {LADDER, PATH}
76 Alors :
77     Si distances[(courant.y + 1) * lvlinfo.xsize
78         + courant.x] = -1
79     Alors :
80         file[fin].x ← courant.x ;
81         file[fin].y ← courant.y + 1;
82         distances[file[fin].y * lvlinfo.xsize
83             + file[fin].x] ← distances[courant.y
84                 * lvlinfo.xsize + courant.x] + 1;
85         predecesseurs[file[fin].y * lvlinfo.xsize
86             + file[fin].x] ← courant;
87         action_precedente[file[fin].y * lvlinfo.xsize
88             + file[fin].x] ← DOWN;
89         fin ← fin + 1;
90     FinSi
91 FinSi
92
93 /* On enfile le sommet de gauche si possible */
94 Si lvlinfo.map[courant.y][courant.x - 1] ∉ {WALL, FLOOR}
95 Alors :
96     Si distances[courant.y * lvlinfo.xsize + courant.x
97         - 1] = -1
98     Alors :
99         file[fin].x ← courant.x - 1;
100        file[fin].y ← courant.y;
101        distances[file[fin].y * lvlinfo.xsize +
102            file[fin].x] ← distances[courant.y
103                * lvlinfo.xsize + courant.x] + 1;
104        predecesseurs[file[fin].y * lvlinfo.xsize
105            + file[fin].x] ← courant;
106        action_precedente[file[fin].y * lvlinfo.xsize
107            + file[fin].x] ← LEFT;
108        fin ← fin + 1;
109    FinSi
110 FinSi
111
112 /* On enfile le sommet de droite si possible */
113 Si lvlinfo.map[courant.y][courant.x + 1] ∉ {WALL, FLOOR}
114 Alors :

```

```

115         Si distances[courant.y * lvlinfo.xsize + courant.x
116             + 1] = -1
117         Alors :
118             file[fin].x ← courant.x + 1;
119             file[fin].y ← courant.y;
120             distances[file[fin].y * lvlinfo.xsize +
121                 file[fin].x] ← distances[courant.y
122                     * lvlinfo.xsize + courant.x] + 1;
123             predecesseurs[file[fin].y * lvlinfo.xsize
124                 + file[fin].x] ← courant;
125             action_precedente[file[fin].y * lvlinfo.xsize
126                 + file[fin].x] ← RIGHT;
127             fin ← fin + 1;
128         FinSi
129     FinSi
130
131     FinTantQue
132
133     /* Retourner les résultats */
134     infopl.dist ← distances;
135     infopl.pred ← predecesseurs;
136     infopl.actionpred ← action_precedente;
137     Retourner infopl;
138 Fin

```

3.2.2 Recherche du bonus le plus proche

```

1 Fonction plus_proche_bonus en bonus
2 Déclaration
3     Enregistrements
4         bonus(x en entier, y en entier),
5         bonus_list(b en bonus, next en pointeur vers bonus_list)
6         levelinfo(map en tableau à 2 dimensions de
7             caractères, xsize en entier, ysize en entier, xexit en
8             entier, yexit en entier),
9
10     Variables
11         ppbonus en bonus, sortie en bonus, min en entier
12
13     Paramètres
14         bonusl en bonuslist, lvlinfo en levelinfo, distances
15         en pointeur vers tableau d'entiers
16 Début
17     /* S'il n'y a aucun bonus dans la liste, alors il faut
18     retourner la sortie */

```

```

19  Si bonusl = NULL Alors :
20      bonus sortie ← {x : lvlinfo.xexit, y : lvlinfo.yexit};
21      Retourner sortie;
22  /* Sinon on cherche quel bonus est le plus proche, et on
23  le retourne */
24  Sinon :
25      ppbonus ← bonusl->b;
26      min ← distances[ppbonus.y * lvlinfo.xsize + ppbonus.x];
27      TantQue bonusl ≠ NULL Faire :
28          Si distances[bonusl->b.y * lvlinfo.xsize +
29          bonusl->b.x] < min
30              Alors :
31                  ppbonus ← bonusl->b;
32                  min ← distances[ppbonus.y * lvlinfo.xsize
33                  + ppbonus.x];
34              FinSi
35              bonusl ← bonusl->next;
36      FinTantQue
37      Retourner ppbonus;
38  FinSinon
39  Fin

```

3.2.3 Reconstruction du chemin

```

1  Procédure reconstruction
2  Déclaration
3      Enregistrements
4          plvalue(dist en tableau d'entiers, pred en tableau de
5              positions, actionpred en tableau d'actions),
6          bonus(x en entier, y en entier),
7          position(x en entier, y en entier),
8
9      Énumérations
10         action={NONE, UP, DOWN, LEFT, RIGHT, BOMB_LEFT,
11             BOMB_RIGHT}
12
13     Variables
14         courant en entier
15
16     Paramètres
17         infopl en pointeur vers plvalue,
18         ppbonus en bonus,
19         posjoueur en entier,
20         x en entier
21  Début

```

```

22   courant ← ppbonus.y * x + ppbonus.x;
23   TantQue infopl->pred[courant].y * x
24       + infopl->pred[courant].x ≠ posjoueur
25   Faire :
26       courant ← infopl->pred[courant].y * x +
27           infopl->pred[courant].x;
28   FinTantQue
29   Retourner infopl->actionpred[courant];
30 Fin

```

3.3 Problèmes rencontrés

Lors de la mise en place de la stratégie, diverses configurations ont mené à des boucles infinies. Les différents points de la stratégie, évoqués partie 3.1, ont été implémentés de sorte à résoudre au fur et à mesure les cas de boucles infinies.

La figure 2 montre par exemple une configuration qui a mené à une boucle infinie. En effet, le joueur regarde s'il y a un ennemi au dessus de lui. C'est le cas, même si en réalité il n'est plus une menace. Donc le programme va lui dire de descendre. Or il y a un ennemi en dessous, donc le programme va lui dire de monter. Ce problème a été résolu notamment en donnant la possibilité de sauter d'une échelle.

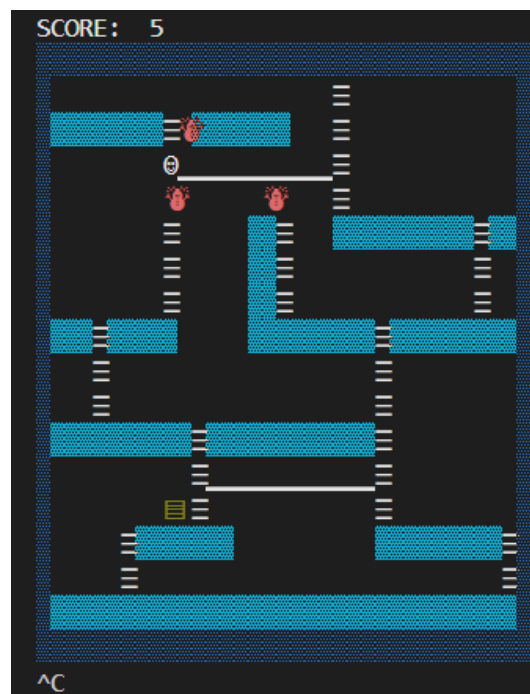


FIGURE 2 – Exemple d'une boucle infinie sur une échelle

Néanmoins, certains cas de boucles infinies n'ont pas pu être résolus, car aucune action ne permet de gagner dans ces configurations. De ce fait, le joueur n'a pas d'échap-

patoire et va bloquer le jeu. Pour palier cela, j'ai mis en place un compteur afin de détecter les boucles infinies. Si une telle boucle est détectée, alors le joueur effectuera l'action NONE. En effet, il est préférable de perdre que de bloquer le jeu. De plus, le fait d'attendre peut mener le joueur à se retrouver dans une situation plus favorable. Par exemple, s'il est bloqué en haut d'une échelle et ne peut pas sauter parce qu'il y a des ennemis dans la zone d'atterrissage, le fait d'attendre va lui permettre de dégager la zone pour sauter, car les ennemis vont monter l'échelle.

La figure 3 montre une autre configuration menant à une boucle infinie. Il y a un ennemi qui arrive par une échelle à gauche, donc le programme dit au joueur d'aller à droite. Or il y a un ennemi à droite, donc le programme lui dit d'aller à gauche.

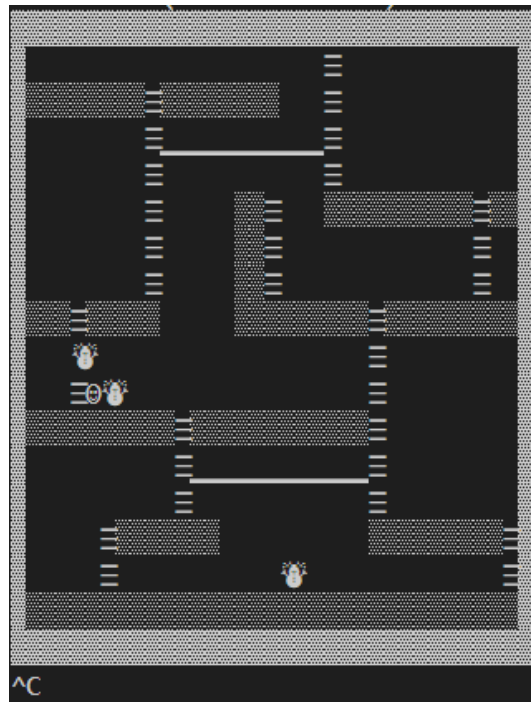


FIGURE 3 – Exemple d'une boucle infinie au pied d'une échelle

La figure 4 montre une seconde configuration menant à un blocage et ne pouvant être résolue. En effet, un ennemi est au dessus, donc le programme va dire au joueur de descendre. Or il y a un ennemi en dessous, donc le programme va lui dire de monter. C'est là qu'intervient une des fonctionnalités du module echelle. Celui ci va détecter qu'il y a un ennemi au dessus et en dessous. Dans ce cas on va chercher sauter. Cependant, le joueur est entouré par le sol de la plateforme. Il ne peut donc pas sauter. Dans ce cas le joueur effectuera NONE et perdra plutôt que de bloquer le jeu.

En plus des problèmes de boucles infinies, j'ai rencontré quelques problèmes liés aux priorités. En effet, le programme ne choisissait pas forcément de gérer l'ennemi le plus dangereux en premier.

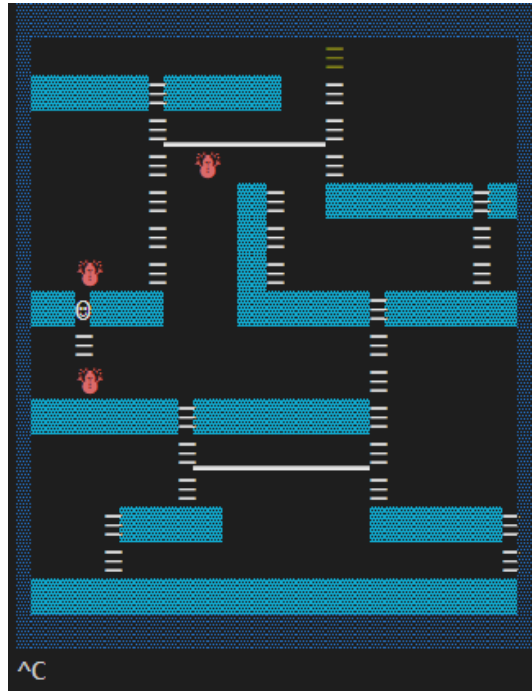


FIGURE 4 – Exemple d’une boucle infinie sur une échelle

Par exemple, lorsque deux ennemis se suivaient exactement (Figure 5), le joueur essayait d’éliminer l’ennemi le plus loin avec une bombe au lieu de fuir celui juste à côté de lui et pourtant plus dangereux. Ce problème a été résolu en donnant la priorité aux ennemis directement à côté du joueur, puis à ceux avec une case de distance.

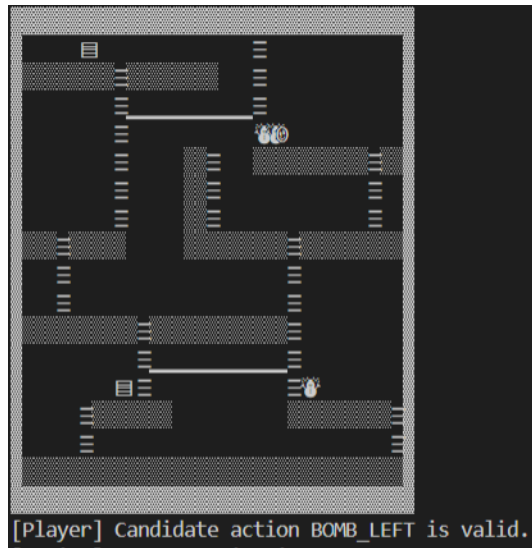


FIGURE 5 – Exemple d’une mauvaise gestion des ennemis

3.4 Évaluation expérimentale

Une fois la phase d'implémentation et de correction des bugs terminée, j'ai exécuté le programme 1000 fois pour chaque niveau afin de tester l'efficacité de ma stratégie. J'estime que 1000 exécutions sont relativement suffisantes pour estimer l'efficacité de mon IA. De plus, cela prend quelques minutes de réaliser autant d'exécutions. Pour ce faire, j'ai utilisé un script python.

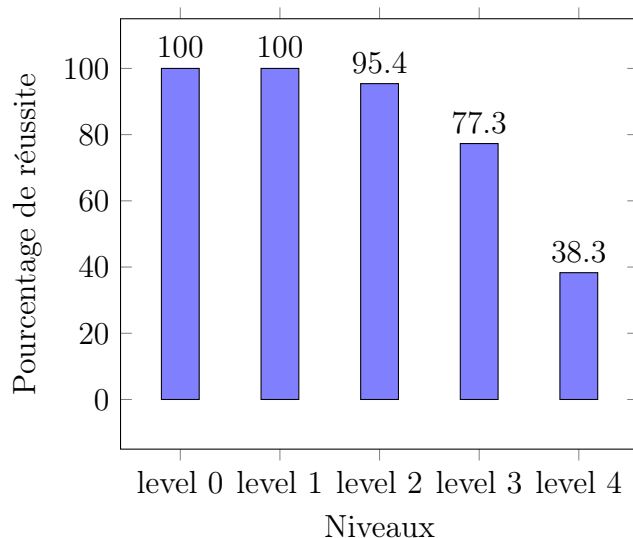


FIGURE 6 – Estimation de la réussite sur les différents niveaux

On constate que la stratégie fonctionne très bien sur les trois premiers niveaux. A partir du niveau 3, les performances sont moins bonnes, mais le joueur gagne toujours plus qu'il ne perd (77.3%). Cependant, le niveau 4 marque un tournant. A partir de ce niveau de difficulté, l'IA n'est plus assez "forte" pour maintenir un nombre de victoires supérieur au nombre de défaites. Cela peut s'expliquer par le fait que les différents paramètres choisis pour la gestion des ennemis ont été déterminés par rapport à la carte du niveau 3. Or le niveau 4 a une disposition différente. Il est à noter que certaines fonctionnalités, implémentées vers la fin du projet, ont réduit les performances sur le niveau 4, mais amélioré celles sur le niveau 3. Malheureusement, je n'ai pas recueilli de données sur cela. Il s'agit simplement d'une constatation non-chiffrée.

4 Conclusion

Pour conclure, ma contribution à ce projet repose sur une stratégie de prise en compte des ennemis lorsqu'ils s'avèrent dangereux. C'est-à-dire que le joueur progresse sans les prendre en compte et essaie de s'en débarrasser quand il arrive face à eux. Il aurait pu être intéressant, voire plus efficace, de calculer des chemins pour éviter les ennemis. Par exemple rajouter un certain nombre à la distance vers le bonus, pour chaque ennemi sur le chemin. Ainsi, il choisirait un chemin peut-être plus long, mais plus sûr. En effet, le problème principal de mon implémentation est qu'elle n'a pas assez de profondeur dans son analyse. De ce fait, le joueur se fait parfois coincer sans option gagnante. Peut-être qu'une méthode de backtracking pourrait régler ce problème. Il faudrait alors définir une profondeur de par exemple 5 tours, et réaliser ce backtracking en s'arrêtant à la première branche trouvée, permettant de ne pas être perdant dans 5 tours. Dès lors, il faudrait jouer la première action de cette branche. On pourrait peut-être le faire sur une partie entière, mais en plus de nécessiter beaucoup de ressources pour le faire une fois, il faudrait le refaire à chaque tour, car nous ne sommes pas autorisés à mémoriser des informations, et si nous éliminons un ennemi, il réapparaîtra en changeant la configuration. Cela nuirait probablement à la fluidité du jeu. De plus, cette solution utilisant un système de backtracking me paraît difficile à implémenter, car il ne faut pas oublier que le joueur doit récupérer les bonus, et non pas simplement survivre. Il faudrait réussir à coordonner la recherche des bonus à la survie par le backtracking.